

KUPC

Kathmandu University Placement Connect

Backend Specification Document

Secure, Real-Time, AI-Assisted Recruitment Platform Backend

Version 2.0 | Engineering Specification | Confidential Draft

Node.js · TypeScript · Express · PostgreSQL · Supabase · OpenAI

Document Control

Field	Value
Project	KUPC — Kathmandu University Placement Connect
Component	Backend (API, data layer, real-time services)
Version	2.0 (expanded from v1.0 roadmap)
Status	Draft — ready for sprint planning
Primary stack	Node.js, Express.js, TypeScript, PostgreSQL (Supabase), OpenAI API
Audience	Backend engineers, DevOps, technical reviewers, hackathon judges

Table of Contents

- 1. Project Overview & Goals
- 2. Technology Stack
- 3. System Architecture
- 4. User Roles & Permissions
- 5. Database Design & Schema
- 6. Authentication & Security Model
- 7. Core API Modules (Endpoint Specification)
- 8. AI Integration (Resume Analysis & Recommendations)
- 9. Real-Time Systems (Chat, Presence, Notifications)
- 10. Swipe & Matching Engine Logic
- 11. Company Verification Workflow
- 12. Analytics & Dashboards
- 13. Non-Functional Requirements
- 14. Testing Strategy
- 15. Deployment & Infrastructure
- 16. Development Roadmap & Phases
- 17. Future Enhancements

1. Project Overview & Goals

KUPC (Kathmandu University Placement Connect) is a Tinder-style recruitment platform exclusively for Kathmandu University students and verified employers. The backend is the system of record for identity, matching, communication, and hiring analytics across the platform, and must be secure, real-time, and horizontally scalable from day one.

1.1 Core Capabilities

- Secure authentication scoped to verified KU email domains, with role-based authorization
- Company verification workflow with document review and admin approval
- Bidirectional swipe-based matching engine (student ↔ company)
- Real-time chat with typing indicators, read receipts, and presence
- AI-powered resume parsing, scoring, and job/candidate recommendations
- Job posting, discovery, search, and filtering
- Multi-channel notifications (in-app, email, future push)
- Analytics for students, companies, and platform admins
- Admin moderation: reports, suspensions, content review

1.2 Design Principles

- **Security first:** RLS at the database layer, not just application-layer checks
- **Statelessness:** JWT-based auth so any API instance can serve any request
- **Event-driven real-time:** Supabase Realtime channels instead of custom WebSocket infra
- **Modular monolith:** one deployable service, cleanly separated by domain module, structured so modules can be extracted into microservices later without a rewrite
- **AI as a service boundary:** all OpenAI calls isolated behind an internal ai/ service layer with retries, timeouts, and cost/usage logging

2. Technology Stack

Layer	Technology	Purpose
Language	TypeScript (strict mode)	Type safety across controllers, services, DB models
Framework	Node.js 20 LTS + Express.js	HTTP API layer, middleware pipeline
Database	PostgreSQL 15 (Supabase-managed)	Relational data store, RLS-enforced
ORM	Prisma ORM (recommended)	Type-safe schema, migrations, query builder
Auth	Supabase Auth (GoTrue)	JWT issuance, refresh tokens, email OTP
Storage	Supabase Storage	Resumes, images, company docs, certificates
Real-time	Supabase Realtime (Postgres CDC)	Chat, presence, typing, live notifications
AI	OpenAI API (GPT-4 class + embeddings)	Resume parsing/scoring, recommendations
Cache / Queue	Redis	Session cache, rate limiting, background job queue
Job Scheduler	BullMQ (Redis-backed)	Async resume analysis, notification delivery

Layer	Technology	Purpose
Validation	Zod	Request schema validation at the edge
Logging	Pino + Sentry	Structured logs, error tracking, audit trail
API Docs	Swagger / OpenAPI 3.1	Auto-generated interactive API reference
Testing	Jest + Supertest	Unit, integration, and API contract tests
Containerization	Docker + docker-compose	Local parity, CI builds, deployment images

ORM decision: Prisma is recommended over the raw Supabase JS SDK for anything beyond simple CRUD, because it gives compile-time-checked queries, migration history, and a single schema source of truth. The Supabase SDK is still used directly for Auth, Storage, and Realtime subscriptions, which Prisma does not cover.

3. System Architecture

3.1 High-Level Flow

Client (web/mobile) → Express API Gateway (auth, rate limit, validation middleware) → Domain Controllers → Service Layer → Prisma / Supabase SDK → PostgreSQL. Real-time events (new message, new match, swipe result) are emitted through Supabase Realtime channels that clients subscribe to directly, bypassing the API for read-path latency. Background work (resume analysis, email delivery, recommendation refresh) is pushed onto a Redis-backed BullMQ queue and processed by dedicated worker processes so the request/response cycle stays fast.

3.2 Folder Structure (expanded)

```
backend/src/
  controllers/ - thin HTTP handlers, no business logic
  routes/ - Express routers per module, versioned under /api/v1
  middleware/ - auth, RBAC, rate-limit, validation, error handler
  services/ - business logic, orchestrates repositories + external APIs
  database/ - Prisma client, repository pattern wrappers
  models/ · types/ · validators/ - Zod schemas + generated Prisma types
  sockets/ - Realtime channel subscription helpers
  ai/ - OpenAI client wrapper, prompt templates, scoring logic
  storage/ · analytics/ · admin/ · jobs/ · notifications/ · chat/ · swipe/ · resume/ · profile/ ·
auth/ - domain modules
  workers/ - BullMQ consumers for async tasks
  config/ - env loading, constants, feature flags
  utils/ - shared helpers (pagination, error classes, response envelope)
  tests/ - unit, integration, e2e, fixtures
```

3.3 Request Lifecycle

- Helmet security headers + CORS allow-list applied globally
- Rate limiter (Redis-backed, per-IP and per-user) rejects abuse before hitting business logic
- JWT verification middleware attaches req.user with id, role, verification status
- RBAC middleware checks route-level required role(s)
- Zod schema validates and strips unknown fields from the request body/query
- Controller calls service layer; service layer is the only place allowed to touch Prisma
- Standard response envelope: { success, data, error, meta }
- Centralized error handler maps thrown AppError instances to HTTP status + code

4. User Roles & Permissions

Role	Key Permissions	Access Scope
Student	Register (KU email only), verify OTP, upload/replace resume, edit profile, swipe companies, chat with matches, save jobs, receive notifications	Own records only; RLS enforced on student_id
Company	Register, submit verification docs, await approval, create/edit/publish/archive jobs, swipe students, chat with matches, manage applicants	Own company_id records only; job actions require verified status

Role	Key Permissions	Access Scope
Admin	Approve/reject companies, suspend/delete accounts, review reports, moderate content, view platform-wide analytics, manage settings	Platform-wide; every mutating action is audit-logged

5. Database Design & Schema

PostgreSQL via Supabase, managed through Prisma migrations. Every table that stores user-owned data has Row-Level Security enabled with policies scoped to `auth.uid()`. Below is the expanded core schema with key columns; full column-level types live in `schema.prisma`.

users

id (uuid, pk) · email (unique) · role (enum: student/company/admin) · email_verified (bool) · status (enum: active/suspended/deleted) · created_at · updated_at

students

id (fk users) · ku_id · full_name · date_of_birth · gender · phone · graduation_year · degree_program · department · resume_score (int) · profile_completion_pct

companies

id (fk users) · company_name · industry · website · size (enum) · verification_status (enum: pending/approved/rejected/suspended) · verified_at · hr_contact_email

admins

id (fk users) · admin_level (enum: super/moderator) · permissions (jsonb)

student_profiles

student_id (fk) · about · skills (text[]) · education (jsonb[]) · projects (jsonb[]) · experience (jsonb[]) · certificates (jsonb[]) · linkedin_url · github_url · portfolio_url · availability · preferred_location

company_profiles

company_id (fk) · logo_url · banner_url · about · benefits (text[]) · gallery (text[]) · recruiters (jsonb[]) · location

resumes

id (pk) · student_id (fk) · file_url · file_hash · version (int) · uploaded_at · is_active

resume_analysis

id (pk) · resume_id (fk) · resume_score (int) · ats_score (int) · grammar_issues (jsonb) · formatting_issues (jsonb) · missing_keywords (text[]) · missing_skills (text[]) · suggestions (text[]) · analyzed_at · model_version

jobs

id (pk) · company_id (fk) · title · description · salary_min · salary_max · location · employment_type (enum) · skills_required (text[]) · deadline · status (enum: draft/published/archived) · created_at

job_skills

job_id (fk) · skill_name · weight (float) — normalized join table for search/scoring

swipes

id (pk) · swiper_id · swiper_type (enum: student/company) · target_id · direction (enum: left/right/super) · created_at — unique(swiper_id, target_id)

matches

id (pk) · student_id (fk) · company_id (fk) · job_id (fk, nullable) · matched_at · status (enum: active/archived)

conversations

id (pk) · match_id (fk, unique) · created_at · last_message_at

messages

id (pk) · conversation_id (fk) · sender_id · content · attachment_url · is_read · read_at · created_at

notifications

id (pk) · user_id (fk) · type (enum) · title · body · payload (jsonb) · is_read · delivered_via (enum: in_app/email) · created_at

saved_jobs

student_id (fk) · job_id (fk) · saved_at — composite pk

company_requests

id (pk) · company_id (fk) · document_type (enum) · file_url · status (enum: pending/approved/rejected) · reviewed_by (fk admins) · reviewed_at

reports

id (pk) · reporter_id · reporter_type · target_id · target_type · reason · description · status (enum: open/dismissed/actioned) · resolved_by · created_at

analytics

id (pk) · metric_name · dimension (jsonb) · value (numeric) · recorded_at — pre-aggregated rollups written by scheduled workers

sessions

id (pk) · user_id (fk) · device_info · ip_address · created_at · expires_at

refresh_tokens

id (pk) · user_id (fk) · token_hash · expires_at · revoked (bool) · created_at

Indexing notes: btree indexes on all foreign keys, composite index on (swiper_id, target_id) in swipes for O(1) duplicate-swipe checks, GIN index on student_profiles.skills and jobs.skills_required for array containment search, and a partial index on matches(status) where status = 'active' to keep dashboard queries fast as the table grows.

6. Authentication & Security Model

6.1 Student Authentication Flow

- Registration restricted to @ku.edu.np-style institutional email domains
- 6-digit OTP sent via Supabase Auth email provider, expires in 10 minutes, max 5 attempts
- On OTP verification, Supabase issues a JWT access token (15 min TTL) and refresh token (30 day TTL)
- Refresh tokens are rotated on every use and stored hashed in refresh_tokens; reuse of a revoked token invalidates the entire session family
- Forgot-password flow re-uses the OTP mechanism rather than emailed reset links, to avoid link-based phishing vectors

6.2 Company Authentication Flow

- Standard email/password registration plus mandatory verification document upload
- Account created in a pending state — cannot post jobs or swipe until verification_status = approved
- Login succeeds even while pending, but protected endpoints return 403 with a clear PENDING_VERIFICATION error code

6.3 Admin Authentication

- Separate login route, never exposed in public API docs
- Mandatory 2FA (TOTP) for all admin accounts
- Every admin action writes an immutable row to an audit_logs table (actor, action, target, before/after diff, timestamp, IP)

6.4 Security Controls

Control	Implementation
Transport security	TLS everywhere; HSTS enabled at the edge/CDN
Password hashing	bcrypt (cost factor 12) — handled by Supabase Auth
Row-Level Security	Postgres RLS policies keyed to auth.uid() on every user-owned table
Input validation	Zod schemas on every route; reject unknown fields
Rate limiting	Redis token bucket — 100 req/min per user, stricter on auth endpoints (5/min)
SQL injection	Prisma parameterized queries only; no raw string interpolation
XSS	Output encoding on any rendered content; sanitize rich-text fields (job descriptions)
CSRF	Not applicable to token-based API, but SameSite cookies used for refresh token storage
File upload safety	MIME-type allow-list, 10MB cap, filename sanitization, virus scan (future)
Secrets management	Environment variables via a secrets manager, never committed
Audit logging	All admin and moderation actions logged immutably

7. Core API Modules (Endpoint Specification)

All routes are versioned under `/api/v1`. Auth column indicates the minimum role required. Full request/response schemas are generated into Swagger at build time; representative examples are shown below for each module.

7.1 /auth

POST `/api/v1/auth/register/student` *Public*

Register with KU email, triggers OTP email.

Request: { "email": "user@ku.edu.np", "full_name": "..."} }

Response: { "success": true, "data": { "otp_sent": true } }

POST `/api/v1/auth/verify-otp` *Public*

Verify OTP and issue access + refresh tokens.

Request: { "email": "...", "otp": "482913" }

Response: { "data": { "access_token", "refresh_token", "user" } }

POST `/api/v1/auth/refresh` *Refresh token*

Rotate refresh token, issue new access token.

POST `/api/v1/auth/logout` *Authenticated*

Revoke current session's refresh token.

POST `/api/v1/auth/company/register` *Public*

Register company + queue verification.

7.2 /profiles

GET `/api/v1/profiles/students/:id` *Authenticated*

Fetch a student profile.

PUT `/api/v1/profiles/students/me` *Student*

Update own student profile.

PUT `/api/v1/profiles/companies/me` *Company (verified)*

Update own company profile.

7.3 /resume & /resume-analysis

POST `/api/v1/resume/upload` *Student*

Upload PDF resume, queues async analysis job.

Request: multipart/form-data (file)

Response: { "data": { "resume_id", "status": "processing" } }

GET `/api/v1/resume-analysis/:resumeId` *Student (owner)*

Poll or fetch completed analysis result.

Response: { "resume_score": 78, "ats_score": 82, "missing_skills": [...] }

7.4 /jobs

POST /api/v1/jobs*Company (verified)*

Create a job posting (draft).

PATCH /api/v1/jobs/:id/publish*Company (owner)*

Publish a draft job.

GET /api/v1/jobs*Authenticated*

Search/filter jobs with pagination.

Response: ?location=&skills;=&salary;_min=&page;=&limit;=

7.5 /swipes & /matches

POST /api/v1/swipes*Student or Company*

Record a swipe; auto-creates match on mutual right-swipe.

Request: { "target_id", "direction": "right" }**Response:** { "data": { "matched": true, "match_id": "..." } }**GET** /api/v1/matches*Authenticated*

List current user's matches.

7.6 /chat & /messages

GET /api/v1/chat/conversations*Authenticated*

List conversations for current user.

POST /api/v1/messages*Authenticated*

Send a message (also broadcast via Realtime).

PATCH /api/v1/messages/:id/read*Authenticated*

Mark message read, updates read receipt.

7.7 Remaining modules

/notifications, /search, /recommendations, /dashboard, /analytics, /reports, /company-verification, /admin, and /settings follow the same envelope, auth, and pagination conventions and are fully enumerated in the generated Swagger spec at /api/v1/docs.

8. AI Integration

8.1 Resume Analysis Pipeline

- Upload triggers a BullMQ job rather than a synchronous OpenAI call, so the upload request returns immediately
- Worker extracts raw text (pdf-parse), then sends a structured-output prompt to the OpenAI API requesting resume score, ATS score, grammar/formatting issues, missing keywords, missing skills, and improvement suggestions as strict JSON
- Response is validated against a Zod schema before being written to resume_analysis; malformed model output triggers one retry, then falls back to a rule-based scorer
- Skill extraction feeds directly into the recommendation engine's candidate vector

8.2 Recommendation Engine

- Hybrid approach: pgvector embeddings (skills, resume text, job description) for semantic similarity, blended with a rules-based score from swipe history and past match outcomes
- Recommendations recomputed on a schedule (nightly) plus incrementally after major profile or resume updates
- Cost control: embeddings cached per resume version; only re-embedded when resume content changes

8.3 Reliability

- All OpenAI calls wrapped with timeout (15s), exponential-backoff retry (max 3), and circuit breaker to avoid cascading failures if the AI provider degrades
- Per-user and platform-wide token usage logged for cost monitoring

9. Real-Time Systems

- **Chat:** messages written to Postgres, then fanned out via Supabase Realtime's Postgres Changes feature — clients subscribe to their conversation_id channel
- **Typing indicators & presence:** ephemeral state via Supabase Realtime Broadcast/Presence channels, not persisted to the database
- **Notifications:** written to the notifications table, then pushed over a per-user Realtime channel; email delivery handled asynchronously by a worker for users who are offline
- **Match updates:** a match event is broadcast to both parties' channels the instant a mutual swipe is detected inside the same database transaction that creates the match row

10. Swipe & Matching Engine Logic

Every swipe is written inside a single transaction that also checks for a reciprocal swipe. This avoids race conditions where both sides swipe within milliseconds of each other.

Pseudocode — swipe transaction

```
BEGIN;
  INSERT INTO swipes (swiper_id, target_id, direction) ON CONFLICT (swiper_id, target_id) DO
  NOTHING;
  IF direction = 'right' THEN
    SELECT 1 FROM swipes WHERE swiper_id = target_id AND target_id = swiper_id AND direction IN
    ('right', 'super');
    IF FOUND THEN
      INSERT INTO matches (...) RETURNING id;
      INSERT INTO conversations (match_id);
      INSERT INTO notifications (...) for both parties;
    END IF;
  END IF;
COMMIT;
```

Business rules

- Self-swipes rejected at the service layer before reaching the database
- Duplicate swipes are idempotent via the unique(swiper_id, target_id) constraint
- Undo is only allowed within a 5-second grace window (optional feature, client-enforced + server-validated by timestamp)
- Super Like bypasses the target's normal recommendation ranking and surfaces the swiper first

11. Company Verification Workflow

Register → Pending → Admin Review → Approve/Reject/Request-More-Docs → Company Activated. Required documents: business registration certificate, PAN/VAT registration, verifiable company website, and an HR point-of-contact email that is validated against the company's own domain where possible.

- Admins can approve, reject with a reason, request additional documents, or suspend an already-approved company
- Every state transition fires a notification to the company and writes an audit log entry
- A company suspended mid-cycle has all active job postings automatically un-published

12. Analytics & Dashboards

Dashboard	Key Metrics
Student	Resume score, recent matches, unread messages, recommended jobs/companies, upcoming interviews, saved jobs, activity timeline
Company	Applicant pipeline, matches, active job posts, response rate, hiring funnel conversion, time-to-hire
Admin	DAU/MAU, total students/companies, pending verifications, open reports, platform growth, swipe/match volume trends

Raw events (swipes, matches, messages, logins) are written to append-only tables and rolled up nightly into the pre-aggregated analytics table so dashboard reads never scan raw event history directly. Chart granularity: daily, weekly, monthly.

13. Non-Functional Requirements

Category	Target
API latency	p95 < 200ms for read endpoints, < 500ms for write endpoints
Availability	99.5% uptime target for MVP; graceful degradation if AI provider is down
Scalability	Stateless API instances behind a load balancer; horizontal scale-out
Data durability	Automated daily Postgres backups, point-in-time recovery via Supabase
Observability	Structured logs, request tracing IDs, error tracking, uptime alerting
Compliance	Data minimization for PII; resumes and documents stored in private buckets with signed, time-limited URLs only

14. Testing Strategy

- **Unit tests:** service layer logic (matching rules, scoring, validation) — Jest
- **Integration tests:** API routes against a test Postgres instance — Supertest
- **Contract tests:** generated Swagger schema validated against actual responses in CI
- **Auth tests:** token expiry, refresh rotation, RBAC boundary cases, RLS policy checks
- **Chat/swipe concurrency tests:** simulate simultaneous mutual swipes to verify no duplicate matches are created
- **Load testing:** k6 scripts targeting swipe and chat endpoints at expected peak concurrency
- **Security testing:** dependency scanning (npm audit / Snyk), basic OWASP ZAP pass pre-release

15. Deployment & Infrastructure

- Dockerized API and worker images, built in CI, pushed to a container registry
- docker-compose for local development (API, worker, Redis, Postgres emulation)
- Managed Supabase project for Postgres, Auth, Storage, and Realtime in staging and production
- CI/CD: GitHub Actions — lint, typecheck, test, build, deploy on merge to main
- Environment separation: local / staging / production, isolated Supabase projects each
- Feature flags for gradually rolling out AI-recommendation and analytics changes

16. Development Roadmap & Phases

Phase	Feature	Priority	Key Deliverables
1	Project Setup & Architecture	High	Express + TS scaffold, folder structure, env config, logging, error handling, security headers, CORS, rate limiting
2	Authentication & Authorization	High	Student/company/admin auth flows, JWT, refresh rotation, RBAC middleware
3	Student & Company Profiles	High	Profile CRUD, storage buckets, validation
4	Resume Upload & AI Analysis	High	Upload pipeline, async worker, OpenAI scoring, resume_analysis persistence
5	Job Posting & Discovery	High	Job CRUD, search/filter, saved jobs
6	Swipe Engine	High	Transactional swipe recording, dedupe, undo/super-like
7	Matching Engine	High	Mutual-swipe detection, match + conversation creation
8	Real-Time Chat	High	Messages, attachments, read receipts, typing, presence
9	Notifications	High	In-app + email delivery, notification preferences
10	Search & Recommendations	Medium	pgvector embeddings, hybrid ranking, nightly refresh
11	Company Verification Workflow	High	Document upload, admin review queue, state machine
12	Dashboard APIs	High	Student/company/admin aggregate endpoints
13	Analytics & Reporting	Medium	Event capture, nightly rollups, chart endpoints
14	Admin Management	High	User/company/job/report management, platform settings
15	Reports & Moderation	Medium	Report submission, review queue, ban/suspend/warn actions
16	Security Hardening	High	RLS policy audit, pen-test fixes, secrets rotation
17	API Documentation	Medium	Swagger/OpenAPI generation, examples, error catalog
18	Testing & Deployment	High	Full test suite, CI/CD, Docker, staging/production rollout

17. Future Enhancements

- AI-powered resume rewriting and cover-letter generation
- AI interview-preparation assistant with mock Q&A
- Skill-gap analysis against target job postings
- Resume version history and rollback
- Push notifications (mobile) and full React Native app
- Interview scheduling with calendar integration
- GitHub / LinkedIn profile import for auto-filled profiles
- University placement office integration, multi-university support
- Employer ratings and reviews, referral system
- Video interview support and coding assessments
- Advanced ML-based recommendation engine trained on platform-wide outcome data

End of document. This specification supersedes v1.0 and should be treated as the reference for sprint planning and API contract work.